
Resilience Matters for Embodied Agents System: New Metrics, Systematic Evaluation, and Optimization

Anonymous Author(s)

Affiliation

Address

email

 Code: EAS_Resilience_Evaluation_Layer

Abstract

Embodied Agents System (EAS) are increasingly deployed in open-world physical domains, where reliability directly dictates deployment quality and human-agent trust. However, existing evaluations rely on outcome-centric metrics as success rate or safety scores that collapse diverse execution trajectories into coarse scores, obscuring the dynamic processes underlying agent behavior. Therefore, they ignore a critical property of EAS – which we define as the **Resilience** – that reflects how EASs *recover*, *stabilize*, and *extend* under perturbations and across iterative updates. The lack of resilience is particularly critical in open-world environments due to continuous unexpected disruptions, thus directly affecting the quality of EAS deployment. To address this problem, we gain insight from the resilience-engineering concepts to EAS groundings and propose a novel resilience evaluation framework that can be flexibly applied to any EAS. Specifically, we define the first comprehensive resilience metrics suite for EASs system that exposes *Rebound*, *Stability*, and *Graceful Extensibility* across embodied tasks execution, providing a practical grounding for EAS resilience analysis. We further implement the resilience evaluation layer that transforms execution evidence into assessments for comparison, diagnosis and optimization. Across 400 household tasks with 10 representative EAS baselines, our evaluation reveals process-level distinction hidden by outcome metrics, including recovery cost differences among successful episodes ($\Delta C_{\text{rec}} = 25.2$), increased instability under semantic perturbations, and task-family-specific degradation under stress. Metrics-guided optimizations reduce recovery cost by 42.94%, increase stability by 19.87%, and improve graceful extensibility completion by 10.08%, showing the diagnostic effect of resilience evaluation. Beyond these targeted gains, our results reveal trade-offs among resilience characteristics, suggesting that a resilient EAS construction should be evaluated and configured according to deployment-specific requirements.

1 Introduction

Embodied Agents System (EAS) shows great potential in domains where physical interaction and long-term task completion are essential, especially for the collaboration domain [21, 4]. Recent advances in large language models (LLMs) further strengthen EAS capabilities of reasoning and interaction, along with increased complexity in reliability evaluation [36, 34, 6]. As illustrated in Fig. 1, two EASs receive the same household instruction and achieve the same outcome, but their thought and execution process can be fundamentally different. This difference matters because EAS operates in dynamic environments where disruptions have continuously emerged [22, 9], indicating that reproducible reliability is critical for physical tasks and collaboration trust during execution [10, 41]. However, current EAS evaluation largely remains outcome or safety metrics,

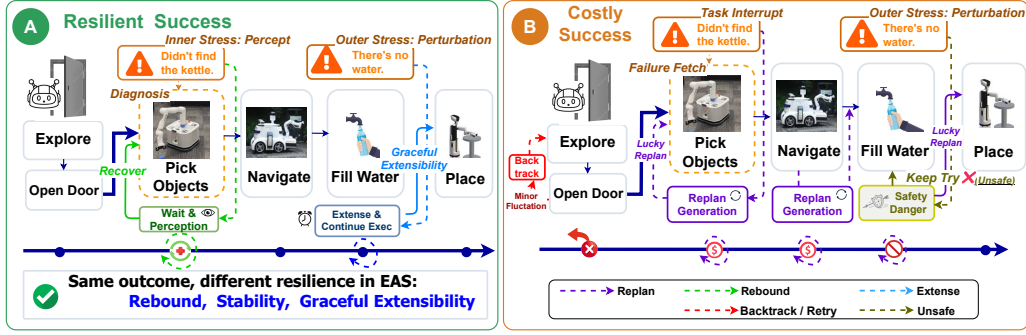


Figure 1: Outcome hide process resilience difference in EAS. (A) A resilient success trajectory handles inner and outer stress through resilient mechanisms, (B) A costly success relies on repeated backtrack, replan and unsafe behaviors.

such as success rate or aggregated safety triggers [6, 40, 19, 34]. They collapse different execution processes into the same score, making brittle success indistinguishable from trustworthy adaptation success [25, 26]. This limitation points to a missing property of EAS – which we define as the **Resilience** in EAS – that reflects how EASs *recover*, *stabilize*, and *extend* under stress.

To address this problem, we gain insights from the resilience-engineering concepts to EAS groundings. System resilience evaluation captures the intrinsic property of a system, which maintains its acceptable execution and effective recovery when unexpected disturbances happen [12, 3]. Woods further decomposes system resilience into four capabilities, including rebound, robustness, graceful extensibility under stress, and sustained adaptability [37]. Specifically, we define the first comprehensive **Resilience Metrics** for EAS that expose *Rebound*, *Stability*, and *Graceful Extensibility* throughout execution: recovery cost C_{rec} , stability sensitivity β , stress response mapping $M_f(\lambda)$ and stress capacity λ_f^* . We further implement the **Resilience Evaluation Layer** that extracts signals from trajectories, monitors, and LLM judge, aggregates them through stage baselines and controlled stress intensities. This layer non-intrusively enables comparison, diagnosis and optimization guidance for EAS resilience evaluation.

We conduct experiments on Habitat-Sim with 400 household tasks and 10 representative EAS methods. Focus on EAS resilience Metrics Validity, Benchmark and Practical Optimization, our results show that resilience evaluation reveals process-level disparities, benchmarking EAS methods and has practical diagnosis effectiveness. Among success episodes, physical perturbations increase recovery cost from 11.6 to 36.3, semantic perturbations raise instability to $\Delta\beta = 0.32$, and stress tests expose degradation, recovery cost and stress intensities are strongly correlated (Spearman $\rho = 0.82$). For methods, Reflexion and CoPAL achieve comparable SR (56.3% vs. 55.8%) but differ by $4.8\times$ in recovery cost, showing that EASs have distinct resilience signatures. We also find that there is usually a trade-off in resilience aspects for EAS method design. Metric-guided optimizations further improve resilience performance, demonstrating that our evaluation layer is both diagnostic and actionable.

In summary, our work makes three major contributions.

1. **Embodied Resilience Metrics and Evaluation Layer:** We formalize resilience metrics as three aspects: *Rebound*, *Stability*, and *Graceful Extensibility*. Then we complement EAS evaluation with the resilience evaluation layer, a practical grounding for measuring EAS resilience property quantification.
2. **Evidence and benchmarking:** We perform a experimental study on 10 representative EAS methods, demonstrating that our metrics successfully distinguish between brittle and resilient behaviors (Sec 3.2.2), we benchmark them, and show their resilience signatures under our resilience evaluation (Sec 3.2.3).
3. **Resilience evaluation provides practical optimization:** We provide quantitative evidence linking specific diagnosis to resilience improvements (Sec 4), showing that resilience-guided diagnosis can support practical optimization, improving EAS resilience while holding the resilience trade-off design principle.

2 Related Works

LLM-Enhanced EAS Evaluation: From Outcome to Resilience Metrics. LLM-enhanced EAS close the perception-execution loop by taking language-based reasoning into embodied actions generation, and they evolved from instruction grounding to feedback reasoning and self-correction [2,

20, 14, 32, 33, 11]. Recent works [15, 23, 43, 35] further explore adaptation, consistency, and open-ended skill acquisition to build trustworthy EAS. However, classical EAS evaluation remains largely outcome-centric as PARTNR emphasizing success metrics [41, 44, 5, 27]. Under embodied scenarios – physical irreversibility, partial observability, and dynamic constraints, such outcomes cannot distinguish robust adaptation from brittle success [21, 6, 34].

Focused on EAS reliability evaluation, recent works construct perturbation test to measure safety-aware planning [40, 38, 7]. They provide reliability diagnostics, but mainly whether an agent violates constraints or approaches a hazardous state in perturbation tests rather than normal process-level execution [19, 45]. We present the resilience metrics to evaluate *EAS Resilience* as an intrinsic property of EAS, measuring how the system behaves from inner stress or perturbations. In this way, we complement EAS evaluation with our resilience evaluation layer to measure how the resilience of EAS exhibits and provide practical optimization guidance.

Resilience Concepts Practical Evaluation in EAS Execution. In systems engineering, resilience emphasizes the capability to endure, adapt to, and recover from disruptions [28, 8]. Woods organizes the technical expression of resilience into four concepts: rebound, robustness, graceful extensibility and sustained adaptability [37]. This perspective holds that handling open-world complexities relies not only on failure prevention, but on continuous, dynamic adaptation during execution [13]. Classical resilience evaluation works evaluate loss severity and recovery latency, but their reliance on exact state matching or crash detection proves insufficient for embodied scenarios [17, 31, 27]. Recently, resilience-relevant evaluating methods for EAS emerged, they evaluate on semantic safety under hazardous instructions and physical cases [46, 42, 22]. However, these methods mix EAS resilience with adversarial security or perturbation stability. They evaluate the failure under static perturbation, but rarely capture the process-aware dynamics like recovery or extensibility.

Grounding with resilient EAS construction, we implement the resilience concepts into three resilient aspects. *Rebound* captures the burden of closed-loop recovery; *Stability* captures robustness as decision consistency; and *Graceful Extensibility* captures bounded degradation and remaining operational margin under stress. Sustained adaptability is treated as a longitudinal extension because it is less directly observable within dynamic EAS execution. Our resilient evaluation realize the practical evaluation layer, providing the validated instrument that reveals the EAS resilience property.

3 The Proposed EAS Resilience Evaluation Framework

Overview. We formalize the Resilience **Metrics** and Resilience **Evaluation Layer** for EAS, which goal is to evaluate system resilience from execution dynamics. Given an episode artifact $(\tau, \mathcal{L}, \mathcal{C}, \mathcal{J})$, where $\tau = \{(o_t, a_t, r_t)\}_{t=1}^T$ is the execution trajectory, $\mathcal{L}$ is the runtime log, $\mathcal{C}$ is the monitor, and $\mathcal{J}$ is the LLM Judge signal, the evaluation layer maps execution dynamics to three resilience aspects:

$$\mathcal{M} : (\tau, \mathcal{L}, \mathcal{C}, \mathcal{J}) \mapsto (\mathcal{M}_{\text{rebound}}, \mathcal{M}_{\text{stability}}, \mathcal{M}_{\text{GE}}).$$

For episode i , let t denote a runtime step. At each step, we record task progress $p_t \in [0, 1]$, proposition completion $q_t \in [0, 1]$, and execution mode m_t . We define a local execution anchor as $a_t := (f, b_t^p, b_t^q, m_t)$, where f is the task family, and b_t^q is the residual proposition load. For each anchor, we estimate a local **Stage Baseline** from clean or reference episodes under the same a_t :

$$\mathcal{N}(a_t) := (\tau^*(a_t), W_{\text{rem}}^*(a_t), \bar{T}^{\text{cog}}(a_t), \bar{N}^{\text{phy}}(a_t), \bar{\Delta}p(a_t), \bar{\Delta}q(a_t), \bar{r}(a_t)).$$

Here, $\tau^*(a_t)$ is the nominal cycle time, $W_{\text{rem}}^*(a_t)$ is the estimated remaining work from the anchor, $\bar{T}^{\text{cog}}(a_t)$ and $\bar{N}^{\text{phy}}(a_t)$ summarize cognitive and physical effort, $(\bar{\Delta}p(a_t), \bar{\Delta}q(a_t))$ summarize task progress and proposition completion, $\bar{r}(a_t)$ summarizes nominal risk load. The Stage Baseline provides a common local reference for resilience metrics computation and the reliability of statistical generalization on resilience metrics. In parallel, we take LLM-as-a-Judge to densify sparse execution feedback. The LLM judges our process from instruction, state, trajectory, and env feedback, and then produces structured reward for goal progress, rational action, and efficiency (detailed in Appendix E.3). Based on this shared evidence substrate, we instantiate system resilience from three complementary aspects: *Rebound*, *Stability*, and *Graceful Extensibility* as Table 1.

3.1 Metrics Design

1) Rebound Metrics: Recovery Cost to Availability. Rebound measures how much extra work the agent must expend to return the acceptable execution after disruption. We use the **Recovery Cost** as the primary indicator, revealing the efficient rebound and brute-force rebound. For detailed

Table 1: Overview of resilience metrics with aspect, type (**D**: deterministic outcome, **S**: statistical, **A**: analysis, **T**: time-series), and data source

Sym.	Metric Name	Calculation	Aspect	Type	Data Source
C_{rec}	Recovery Cost	$C_{\text{rec}} = \frac{1}{W_{\text{rem}}^*(a_{t_d})} \sum_{t=t_d}^{t_r} \frac{\Delta \tau_t}{\tau^*(a_t)} \cdot g_t^{\text{rec}}$	Rebound	D, S	System Logs
β	Policy Stability	$\beta = \sup_{z, \rho} \frac{ \ell(\pi, \rho(z)) - \ell(\pi, z) }{\ \rho\ }$	Stability	S	Sensitivity Test
$M_f(\lambda)$	Stress Curve Mapping	$M_f(\lambda) = \frac{1}{K} \sum_{j=1}^K \tilde{r}_{(j),f}(\lambda) - 1$	Extensibility	D, S	Stress Test
λ_f^*	Stress Capacity	$\lambda_f^* = \sup\{\lambda \in [0, 1] \mid M_f(\lambda) \geq 0\}$	Extensibility	S, A	Stress Grid

implementation, please refer to Appendix A.1. For a rebound window $[t_d, t_r]$, t_d is the first step at which execution departs from the nominal continuation under the local anchor a_t , and t_r is the first step at which the task re-enters the acceptable execution. Execution *StageBaseline* $\mathcal{N}(a_t)$ (execution local reference, refer to Sec 3.2) provides the nominal local reference, including the remaining nominal work $W_{\text{rem}}^*(a_t)$, and the expected local execution statistics.

We decompose recovery cost into three aligned parts: **Cognitive recovery**, **Physical recovery**, and **State debt** in Eq. 1. We aggregate their excess variables as

$$\mathbf{z}_t^{\text{cog}} = \frac{(T_t^{\text{cog}} - \bar{T}^{\text{cog}}(a_t))_+}{\bar{T}^{\text{cog}}(a_t) + \epsilon}, \quad \mathbf{z}_t^{\text{phy}} = \frac{(N_t^{\text{phy}} - \bar{N}^{\text{phy}}(a_t))_+}{\bar{N}^{\text{phy}}(a_t) + \epsilon}, \quad \mathbf{z}_t^{\text{deb}} = \frac{(r_t - \bar{r}(a_t))_+}{\bar{r}(a_t) + \epsilon}. \quad (1)$$

where $\mathbf{z}_t^{\text{cog}}$ measures excess planning, perception effort; $\mathbf{z}_t^{\text{phy}}$ measures excess navigation, retry effort; and $\mathbf{z}_t^{\text{deb}}$ measures lagged progress leakage, risk load. $(\cdot)_+$ indicates the ReLU function, we record the excess costs while discarding pre-complete [1]. Detailed illustration refers to appendix Eq.1. To make the three channels comparable under the same anchor-conditioned regime, we convert each local excess vector into a covariance-adjusted deviation as Mahalanobis Distance[24]:

$$g_t^u = \sqrt{\mathbf{z}_t^{u\top} (\Sigma_u(a_t) + \eta I)^{-1} \mathbf{z}_t^u}, \quad u \in \{\text{cog}, \text{phy}, \text{deb}\}, \quad (2)$$

where $\Sigma_u(a_t)$ is the covariance estimated from standard stage baselines, ηI is the regularizer. This step removes scale inconsistency and accounts for correlation within each recovery channel. We then aggregate the three channel deviations into recovery intensity $g_t^{\text{rec}} = \sqrt{\frac{1}{U} \sum_u (g_t^u)^2}$. Finally, for episode i , the *Rebound Cost* is defined as

$$C_{\text{rec}}^{(i)} = \frac{1}{W_{\text{rem}}^*(a_{t_d}) + \epsilon} \sum_{t=t_d}^{t_r} \frac{\Delta \tau_t}{\tau^*(a_t) + \epsilon} \cdot g_t^{\text{rec}}. \quad (3)$$

This quantity measures the total extra work required to restore acceptable execution after disruption. Normalization by $W_{\text{rem}}^*(a_{t_d})$ ensures that recovery is judged against the remaining nominal work at the disturbance point. We aggregate rebound signature through statistics in Appendix A.1.

2) Stability Metrics: Execution Fluctuations. Stability quantifies the system’s ability to remain *consistent* under stresses, whether arising from stochasticity (Execution), perturbations (Sensitivity), or self-evolution. It is reflected in execution as frequent replanning, unstable progress, and abrupt shifts in the progress. We take *Policy Stability* (β) as the primary indicator, because it directly measures whether small changes in task instruction or execution condition amplify into process fluctuations. With implementation details like value networks refer to Appendix A.2, we describe how β is computed from execution representations.

We use the value function $V(\cdot)$ for each EAS execution representation, which is used to diagnose execution progress changes trend. Given the encoded state s_t , the value function is

$$V(s_t) = \mathbb{E}_{\pi} \left[\sum_{u=t}^T \gamma^{u-t} \tilde{r}_u \mid s_t \right], \quad (4)$$

where π is the agent policy and $\tilde{r}_u$ is the shaped reward by LLM judge and W_{rem}^* . For each state transition, we compute the one-step temporal-difference residual $\delta_t = \tilde{r}_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$, where d_t is the termination signal. We further compute the generalized advantage estimate A_t as an exponentially decayed accumulation of TD residuals: $A_t = \sum_{\ell=0}^{T-t} (\gamma\lambda)^\ell \delta_{t+\ell}$. Then our resilience

163 loss for shape reward update is

$$\ell(\pi, z_i) = \frac{1}{T_i} \sum_{t=1}^{T_i} \sqrt{\mathbf{x}_t^\top (\Sigma(a_t) + \eta I)^{-1} \mathbf{x}_t}, \quad (5)$$

164 where z_i denotes an episode, $\Sigma(a_t)$ is the standard stage baselines covariance, and ηI is regularizer.
 165 This loss increases when the trajectory $\mathbf{x}_t$ contains large value variance, TD spikes, and advantages.
 166 We define β -Stability as this perturbations sensitivity to execution progress:

$$\beta_f = \sup_{z \in \mathcal{Z}_f, \rho \in \mathcal{P}_f} \frac{|\ell(\pi, \rho(z)) - \ell(\pi, z)|}{\|\rho\| + \epsilon}. \quad (6)$$

167 Here, $\mathcal{Z}_f$ is the episode set from task family f , $\mathcal{P}_f$ is the controlled perturbations set. A low β_f
 168 means that similar conditions lead to similar results, while high β_f means more replanning, value
 169 fluctuation, and progress instability.

170 **3) Graceful Extensibility Metrics: Stress Response.** Graceful Extensibility measures how an EAS
 171 performance extends as stress increases. It ensures that when stressors exceed the system’s adaptive
 172 capacity, performance declines *gracefully* and predictably rather than collapsing catastrophically.
 173 This is a dynamic stress-response, revealing the resilience property of EAS. We therefore model
 174 Graceful Extensibility as a stress-response mapping $\lambda \mapsto M_f(\lambda)$, where λ is the stress severity and
 175 $M_f(\lambda)$ is the operational margin.

176 Let $\mathcal{T}_\lambda$ denote a stress operator with severity $\lambda \in [0, 1]$, which is related to perturbations and EAS
 177 self-evolvement. In our resilience evaluation, we unify external perturbation and internal evolution
 178 by the same stress-response in Appendix A.3. For an episode i from task family f under stress $\mathcal{T}_\lambda$,
 179 we first define a **hard** boundary monitor:

$$m_i(\lambda) = \min \left\{ \frac{p_{\mathcal{T},i}(\lambda)}{\tau_p}, \frac{q_{\mathcal{T},i}(\lambda)}{\tau_q}, \frac{\tau_{\text{rec}}}{C_{\text{rec},i}(\lambda) + \epsilon}, \frac{\tau_{\text{stab}}}{\beta_{\text{stab},i}(\lambda) + \epsilon}, \frac{\tau_{\text{safe}}}{U_i(\lambda) + \epsilon} \right\} - 1. \quad (7)$$

180 where U_i is the constraints violation load. The thresholds define the minimal acceptable task contract,
 181 $m_i(\lambda) < 0$ means that the contract is violated.

182 After accumulation of stage baselines from the task family f , we aggregate the statistical extensibility
 183 of the operation. For compact notation, let $Q_\alpha^f[X](\lambda) \triangleq Q_\alpha(X(\lambda) \mid f)$, where $Q_\alpha^f[X](\lambda)$ denotes
 184 the α -quantile of variable X over episodes from family f under stress λ . We construct the statistical
 185 extensibility vector

$$\mathbf{r}_f(\lambda) = \left[\frac{Q_\alpha^f[p_T](\lambda)}{\tau_p}, \frac{Q_\alpha^f[q_T](\lambda)}{\tau_q}, \frac{\tau_{\text{rec}}}{Q_{1-\alpha}^f[C_{\text{rec}}](\lambda) + \epsilon}, \frac{\tau_{\text{stab}}}{Q_{1-\alpha}^f[S_{\text{stab}}](\lambda) + \epsilon}, \frac{\tau_{\text{safe}}}{Q_{1-\alpha}^f[U](\lambda) + \epsilon} \right]^\top. \quad (8)$$

186 We take lower and upper quantiles for benefit variables and cost variables, which makes our metrics
 187 sensitive to graceful extensibility. Let $r_{(1),f}(\lambda) \leq \dots \leq r_{(K_{\text{max}}),f}(\lambda)$ be the sorted entries of $\mathbf{r}_f(\lambda)$.
 188 The statistical operational margin is defined as

$$M_f(\lambda) = \frac{1}{K} \sum_{j=1}^K r_{(j),f}(\lambda) - 1, \quad 1 < K \leq K_{\text{max}}. \quad (9)$$

189 This bottom- K aggregation preserves the semantics of extensibility, but avoids the rigid behavior of a
 190 minimum optimization. For practical diagnosis, dynamic **mapping** between $M_f(\lambda)$ extensibility and
 191 execution performance is an important indicator to analyze EAS resilience. Further, the quantitative
 192 Graceful Extensibility indicator is the largest stress level where the EAS remains acceptable execution:

$$\lambda_f^* = \sup \{ \lambda \in [0, 1] \mid M_f(\lambda) \geq 0 \}. \quad (10)$$

193 We call λ_f^* the *stress capacity*. It gives the operational boundary of task family f : the maximum
 194 stress severity at which the system can still preserve its positive task progress in task family f .

195 3.2 Resilience Evaluation Benchmark

196 **The Resilience Evaluation Layer Construction** We construct the Resilience Evaluation Layer as
 197 a non-intrusive evaluation instrument that complements the EAS reliability evaluation process. We
 198 present our resilience evaluation pipeline in Alg 1, it attaches probes during the EAS execution, then
 199 computes collected signals to resilience metrics. This design allows resilience to be evaluated as an
 200 execution property rather than an additional task objective.

Algorithm 1 Resilience Metrics Evaluation Pipeline

Require: Task family set $\mathcal{F}$; episode artifacts $\{(\tau_i, \mathcal{L}_i, \mathcal{C}_i, \mathcal{J}_i)\}_{i=1}^N$; StageBaseline estimation; perturbation set $\mathcal{P}_f$; stress grid $\Lambda = \{\lambda_j\}_{j=1}^m$; fixed thresholds and windows.

Ensure: $\mathcal{M}_{\text{rebound}}, \mathcal{M}_{\text{stability}}, \mathcal{M}_{\text{GE}}$

- 1: Estimate Stage Baselines $\mathcal{N}(a)$ from executions.
- 2: **for** each task family $f \in \mathcal{F}$ **do**
- 3: **for** each stress level $\lambda \in \Lambda$ **do**
- 4: **for** each episode i , artifact $(\tau_i, \mathcal{L}_i, \mathcal{C}_i, \mathcal{J}_i)$ **do**
- 5: Extract stepwise signals: progress p_t, q_t , execution mode m_t , execution $\mathcal{L}_i$, critics $\mathcal{C}_i, \mathcal{J}_i$.
- 6: Assign local execution anchors $a_t = (f, b_t^p, b_t^q, m_t)$.
- 7: **if** Detect **Rebound** window $[t_d, t_r]$ **then**
- 8: Compute $\mathbf{z}_t^{\text{cog}}, \mathbf{z}_t^{\text{phy}}$ and $\mathbf{z}_t^{\text{deb}}$.
- 9: Compute rebound cost $C_{\text{rec}}^{(i)}$ from g_t^{rec} .
- 10: Return to acceptable $\mathcal{N}(a_t)$.
- 11: **end if**
- 12: Collect **Stability** values $V(s_t)$, TD residuals δ_t , and GAE advantages A_t .
- 13: Aggregate to vector $\mathbf{x}_t$ and β_i .
- 14: Backpropagation Critic $\ell_{\text{stab}}(\pi, z_i)$ and Stage Baselines $\mathcal{N}_f(a)$.
- 15: **end for**
- 16: Retrieve stressed executions under $\mathcal{T}_\lambda$.
- 17: Compute hard boundary monitors $m_i(\lambda)$ in family f .
- 18: Aggregate episode statistics $\mathbf{r}_f(\lambda)$.
- 19: Compute stress mapping: $M_f(\lambda) \leftarrow \frac{1}{K} \sum_{j=1}^K r_{(j),f}(\lambda) - 1$.
- 20: **end for**
- 21: Compute Graceful Extensibility stress capacity: $\lambda_f^* \leftarrow \sup\{\lambda \in \Lambda \mid M_f(\lambda) \geq 0\}$.
- 22: Update Stage Baseline $\mathcal{N}$ and Statistical Aggregation Metrics
- 23: **end for**
- 24: **return** $C_{\text{rec}}, \beta, M_f, \lambda_f^*$, trajectory.

3.2.1 Resilience EAS Testbed Setup

We conduct experiments on *Habitat-Sim 3.0* [29] with PARTNR-style [6] collaborative household tasks, including navigation, object manipulation, and object transportation scenarios. To expose different EAS resilience, we introduce controlled perturbations over physical states, semantic instructions, object states, and stress severity. The full perturbation specification, evaluated EAS methods, and dataset setting are reported in Appendix B.4, Appendix C.2, and Appendix B.2.

3.2.2 Resilience Metrics Validation: Execution Differences Hidden by Outcomes

We first validate whether resilience metrics ($\mathcal{M}_{\text{rebound}}, \mathcal{M}_{\text{stability}}, \mathcal{M}_{\text{GE}}$) capture distinct resilience property hidden by outcome-centric evaluation, and verify them as the measurement instruments.

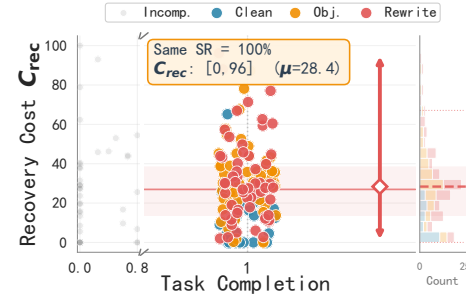


Figure 2: Different C_{rec} under outcomes.

Resilience metrics disperse differences in same outcome.

Resilience metrics reveal significant performance differences that both SR and safety scores fail to capture, demonstrating discrimination capability for EAS with similar success rates and safety scores. We take C_{rec} as an example, Fig. 2 illustrates "the same outcome, different resilience" effect.

Resilience Metrics Construct Validity and Sensitivity.

Fig. 3 further confirms the construct validity of the three metric families. Rebound C_{rec} separates executions by recovery burden: perturbations increase recovery cost from 11.6 in clean episodes to 36.3 in average, with a significant distributional shift (Mann–Whitney $p < 0.001$). Stability β captures sensitivity to semantic-preserving instruction changes: semantic perturbations inflate instability to $\beta = 0.32$, mainly driven by divergent action generation instability ($\beta_{\text{out}} = 0.42$). Graceful Extensibility characterizes degradation as a stress-response curve rather than a binary failure event: as stress severity increases over $\lambda \in [0.2, 1.0]$, the relative margin ΔM decreases while recovery cost rises, with strong correlation (Spearman $\rho = 0.82$). These results show that the metrics measure complementary resilience properties and are not reducible to SR, safety, or completion alone.

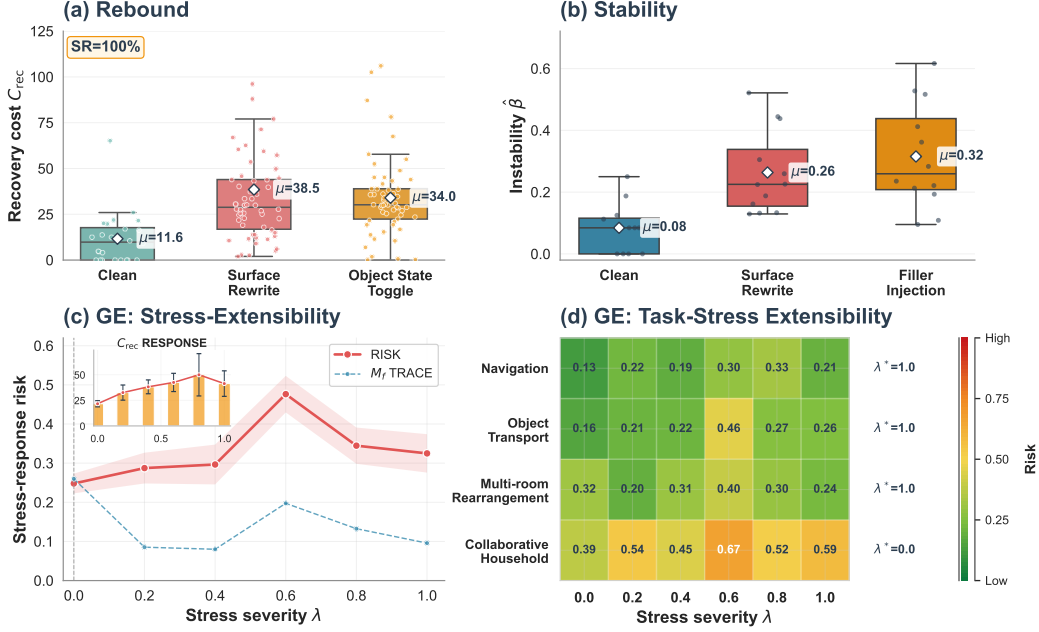


Figure 3: Resilience metrics construct validity results (zoom in for better view).

In addition, we show the statistical reliability and sensitivity of the analysis of resilience metrics in Appendix D.3. This indicates that resilience metrics are related to EAS inherent property, which maintains the consistency across embodied tasks execution.

Also, we conduct scalability sensitivity analysis across Qwen3-8B-Instruct, Qwen2.5-7B-Instruct, and Llama3.1-8B-Instruct shows that stronger backbones improve baseline task performance while preserving similar resilience profile patterns, suggesting that resilience is a system-level property rather than only a backbone-size effect (Appendix D.4).

3.2.3 Benchmarking EAS Resilience Results

We apply the validated resilience metrics to benchmark representative EAS methods under identical environments and perturbation settings. Table 2 reports the resilience benchmark scores, and Fig. 4 summarizes the normalized resilience profiles. For example, Reflexion and CoPAL achieve comparable SR (56.3% vs. 55.8%), but Reflexion incurs $4.8\times$ higher recovery cost than CoPAL (57.3 vs. 11.9). This indicates that EAS take different recovery processes, and resilience evaluation reveals these features.

This benchmark also shows that no current method dominates all resilience aspects. CoPAL achieves a favorable rebound-extensibility trade-off, with low recovery cost ($C_{rec} = 11.9$) and the highest stress capacity ($\lambda^* = 0.85$), but it requires long simulated execution steps. InnerMono has the best stability profile ($\beta = 0.149$), yet its recovery cost remains high ($C_{rec} = 43.5$). AgentEvolver has relatively low recovery cost ($C_{rec} = 19.2$), yet its stress capacity is the weakest among all methods ($\lambda^* = 0.21$), showing limited graceful extensibility. These results demonstrate that resilience should be interpreted as a multi-dimensional execution profile rather than a single scalar ranking.

EAS Methods Resilience Signatures Under the same benchmark protocol, we analyze distinct behaviors (performance, simulation steps, rebound, stability, and graceful extensibility) among the methods in Fig. 5. The shape of the radar plot serves as a visual fingerprint for the EAS resilience profile, and exhibits different resilience profiles, strength, and weakness. Some methods continue

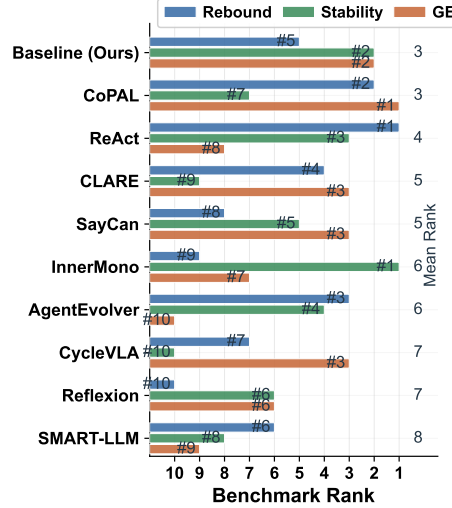


Figure 4: Resilience benchmark landscape.

Table 2: Comparative EAS resilience benchmark results in resilience aspects.

Method	Resilience Metrics			Outcome Ref.		Sim. Steps
	$C_{rec} (\downarrow)$	$\beta (\downarrow)$	Stress Cap. $\lambda^* (\uparrow)$	SR	Comp.	
ReAct	6.5 ± 0.46	0.239 ± 0.011	0.50 ± 0.042	$38.5\% \pm 3.1\%$	$57.8\% \pm 4.6\%$	3374 ± 285
Reflexion	57.3 ± 5.07	0.299 ± 0.021	0.66 ± 0.035	$56.3\% \pm 4.2\%$	$75.0\% \pm 3.8\%$	4390 ± 312
CycleVLA	35.5 ± 2.91	0.400 ± 0.026	0.72 ± 0.047	$22.2\% \pm 2.8\%$	$47.5\% \pm 4.1\%$	4688 ± 420
CLARE	21.6 ± 1.68	0.344 ± 0.016	0.72 ± 0.052	$28.4\% \pm 3.5\%$	$46.0\% \pm 4.5\%$	4249 ± 360
SayCan	38.5 ± 2.49	0.283 ± 0.013	0.78 ± 0.046	$57.0\% \pm 4.1\%$	$72.6\% \pm 3.7\%$	4682 ± 380
AgentEvolver	19.2 ± 1.24	0.264 ± 0.012	0.21 ± 0.015	$48.8\% \pm 3.9\%$	$66.8\% \pm 4.2\%$	3084 ± 250
InnerMono	43.5 ± 2.69	0.149 ± 0.007	0.53 ± 0.026	$49.5\% \pm 4.0\%$	$67.5\% \pm 4.3\%$	3670 ± 290
CoPAL	11.9 ± 1.02	0.302 ± 0.017	0.85 ± 0.078	$55.8\% \pm 4.5\%$	$72.6\% \pm 3.9\%$	4846 ± 410
SMART-LLM	28.2 ± 2.20	0.339 ± 0.018	0.40 ± 0.036	$55.7\% \pm 3.8\%$	$74.3\% \pm 4.1\%$	5085 ± 395
Baseline	25.0 ± 2.03	0.200 ± 0.009	0.79 ± 0.042	$53.0\% \pm 3.6\%$	$69.7\% \pm 3.5\%$	3671 ± 275

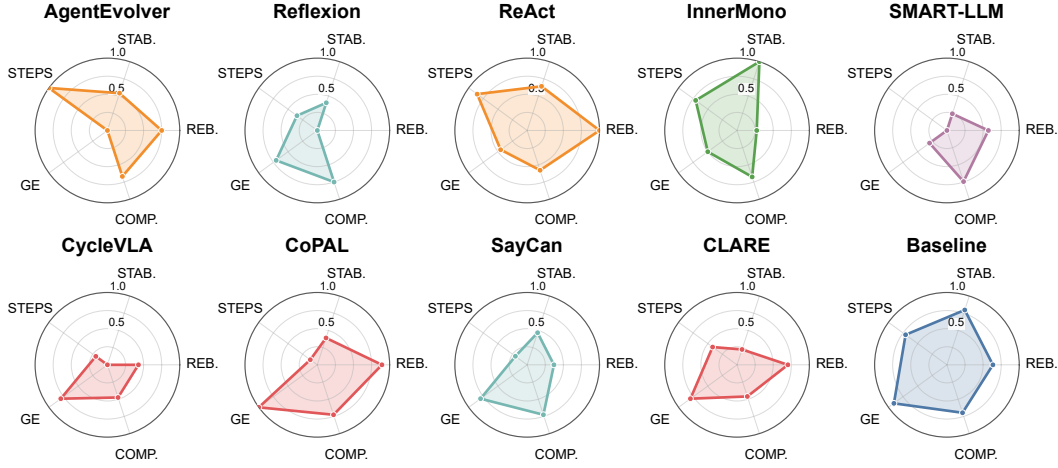


Figure 5: The Resilience Diagnosis. Visualizing the distinct behavioral archetypes of EASs.

execution through costly recovery, some maintain stable execution, some fluctuate during planning, and some show graceful extensibility under stress. These profiles provide a resilience overview of current EAS methods and serve as a diagnostic handbook for optimization experiments. We conclude resilience benchmark findings as below:

Resilience Evaluation Benchmark Findings

- 1) Outcome-equivalent executions can be resilience-inequivalent.** Success rate and completion describe whether an EAS finishes the task, but they do not reveal how much recovery, replanning instability, or stress margin is consumed during execution. Resilience metrics expose these hidden process differences.
- 2) There is no current EAS method dominates all resilience aspects.** Some methods recover efficiently, some remain stable under semantic variation, and some tolerate stress, but no method dominates all dimensions. This indicates that EAS deployment should be matched to the target failure regime: difficult tasks require stronger Graceful Extensibility, while sensitive tasks require stronger Stability.
- 3) Resilience metrics provide actionable diagnosis.** Abnormal metric patterns can be traced back to concrete execution events, such as repeated recovery loops, action oscillation, or stress-boundary collapse. This makes the evaluation layer not only a benchmark tool, but also a diagnostic basis for the metric-guided optimizations in Sec. 4.

4 EAS Optimization: Metric-Guided Resilience Diagnostics

Based on the resilience metrics and the EAS resilience profiles, we introduce a unified *metric-guided resilience optimization* to the EAS execution. The goal is not to replace the original planner, but to use our resilience metrics as diagnostic signals that indicate where the execution loop should be minimally repaired. We implement three targeted optimizations for Rebound, Stability, and Graceful Extensibility, with detailed implementation in Appendix C.5. We compare the *Original* and *Optimized* variants under identical evaluation conditions. Table 3 shows that the resilience metrics are not only descriptive, but can also guide practical resilience improvements. What’s more, targeted optimization

Table 3: Comparative results of metric-guided targeted improvements.

Target	Methods	Rebound $C_{\text{rec}} \downarrow$	Rec. Win. $\downarrow$	$\beta_{\text{step}} \downarrow$	$\beta \downarrow$	GE. Comp. $\uparrow$	Formal GE
Rebound	Baseline	59.11	3.13	0.2881	0.6834	0.887	–
	Optimized	33.73 $\downarrow 42.94\%$	2.38 $\downarrow 23.96\%$	0.2079 $\downarrow 27.84\%$	0.6870 $\uparrow 0.53\%$	0.819 $\downarrow 7.67\%$	–
Stability	Baseline	51.01	2.63	0.3185	0.6788	0.903	–
	Optimized	43.01 $\downarrow 15.68\%$	3.00 $\uparrow 14.07\%$	0.2552 $\downarrow 19.87\%$	0.6772 $\downarrow 0.24\%$	0.750 $\downarrow 16.94\%$	–
GE	Baseline	36.70	1.33	1.4594	0.6917	0.833	Incomplete
	Optimized	99.59 $\uparrow 171.36\%$	2.25 $\uparrow 69.17\%$	0.7215 $\downarrow 50.56\%$	0.6371 $\downarrow 7.89\%$	0.917 $\uparrow 10.08\%$	Complete [†]

[†] Formal GE is reported only for GE stress-tests, since it requires a stress-response curve over λ grid.

of a single resilient aspect often leads to compromises in other resilience aspects (optimizing GE results in high recovery cost C_{rec}). This confirms our findings: resilience is a multi-dimensional structural balance, and our resilient evaluation layer could capture this trade-off.

Opt1 Recovery: Feedback-Guided Recovery Implementation High recovery cost C_{rec} and long recovery windows indicate that the agent often spends extra reasoning or physical actions after local failures. To address this failure mode, we add a feedback-guided recovery layer on top of the centralized LLM planner. The layer monitors World-Graph evidence, recent execution feedback, and task-progress stagnation, and diagnoses recoverable faults such as missing graph nodes, stale object locations, repeated tool failures, or invalid object references. Instead of overriding the planner with a separate recovery policy, we get *Rebound Guidance* Π_{t+1} into the next planning context, including perception refresh, reflection, state summarization, or belief rollback. Then we formalize it as Habitat skill tools for better use. This keeps the original action space unchanged while converting low-level execution feedback $\mathcal{F}_t$ into planner recovery hints. As shown in Table 3, this optimization reduces C_{rec} from 59.11 to 33.73 and the recovery window from 3.13 to 2.38, showing that the metric-guided feedback loop directly reduces redundant recovery effort.

Opt2 Stability: Consistency State Record High β values reveal unstable replanning behavior, where the agent adapts to new feedback but may oscillate between similar actions, repeat completed subtasks, or drift away from the current execution c_t . We therefore introduce a consistency record for replanning. At each step, it constructs a read-only phase guidance from the task goal, current phase, World-Graph state, current room and holding state, completed and pending subtasks, and recent action-response history. This design improves consistency without making the policy brittle. The optimized variant reduces β_{step} from 0.3185 to 0.2552 and improves β from 0.6788 to 0.6772, indicating that the stability metric captures and guides reductions in local replanning variance.

Opt3 GE: Boundary Long-tail Control for Difficulties Graceful Extensibility focuses on whether performance degrades smoothly as stress increases, rather than collapsing or entering unbounded long-tail behavior. In high-stress cases, we observed that the planner may repeatedly wait or receive near-duplicate execution feedback without making progress. Such behavior inflates execution cost and makes the stress-response curve insensitive to true adaptive capacity. To control this boundary effect, we add a long-tail monitor that detects dual-wait patterns, semantically repeated feedback, and stagnant task progress. When a boundary stall B_t is detected, the system injects a bounded cognitive reset that discourages further non-progressive waiting and requests either a concrete productive action or termination. This converts uncontrolled long-tail execution into measurable boundary execution.

5 Conclusion and Future Work

We presented the first Resilience Evaluation Framework for EASs. By designing novel metrics: *Rebound*, *Stability*, and *Graceful Extensibility*, our framework can expose resilience hidden pathologies, such as high-cost recovery and catastrophic forgetting, which remain invisible to outcome-centric metrics. Empirical analysis across representative baselines identified distinct Resilience Archetypes, confirming that architectural choices involve quantifiable trade-offs between execution, stability, and adaptability. The proposed optimization framework can help EASs developers to diagnose root causes and enforce non-degradation contracts to build a resilient and trustworthy EAS.

Future Work Future work can extend resilience evaluation along three directions: *Sim2Real* validation of whether simulation-derived resilience profiles transfer to physical robots, *theoretical analysis of trade-offs* among resilience aspects, and *deployment-aware optimization* that uses benchmarked resilience signatures to improve EASs for target application conditions. These directions can move resilience evaluation toward more principled, deployable, and balanced EAS design.

References

- [1] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [3] Algirdas Avizienis, J-C Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE transactions on dependable and secure computing*, 1(1):11–33, 2004.
- [4] Rajkumar Buyya et al. Agentic artificial intelligence (ai): Architectures, taxonomies, and evaluation of large language model agents. *arXiv preprint arXiv:2601.12560*, 2026.
- [5] Matthew Cavorsi, Lorenzo Sabattini, and Stephanie Gil. Multirobot adversarial resilience using control barrier functions. *IEEE Transactions on Robotics*, 40:797–815, 2023.
- [6] Matthew Chang, Gunjan Chhablani, Alexander Clegg, Mikael Dallaire Cote, Ruta Desai, Michal Hlavac, Vladimir Karashchuk, Jacob Krantz, Roozbeh Mottaghi, Priyam Parashar, et al. Partnr: A benchmark for planning and reasoning in embodied multi-agent tasks. *arXiv preprint arXiv:2411.00081*, 2024.
- [7] Zixing Chen, Yifeng Gao, Li Wang, Yunhan Zhao, Yi Liu, Jiayu Li, Xiang Zheng, Zuxuan Wu, Cong Wang, Xingjun Ma, et al. Hazardarena: Evaluating semantic safety in vision-language-action models. *arXiv preprint arXiv:2604.12447*, 2026.
- [8] Richard I Cook and Beth Adele Long. Building and revising adaptive capacity sharing for technical incident response: A case of resilience engineering. *Applied ergonomics*, 90:103240, 2021.
- [9] Pascale Fung, Yoram Bachrach, Asli Celikyilmaz, Kamalika Chaudhuri, Delong Chen, Willy Chung, Emmanuel Dupoux, Hongyu Gong, Hervé Jégou, Alessandro Lazaric, et al. Embodied ai agents: Modeling the world. *arXiv preprint arXiv:2506.22355*, 2025.
- [10] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025.
- [11] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12124–12131. IEEE, 2024.
- [12] Crawford S Holling et al. Resilience and stability of ecological systems, 1973.
- [13] Erik Hollnagel. *Safety-I and safety-II: the past and future of safety management*. CRC press, 2018.
- [14] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- [15] Frank Joublin, Antonello Ceravola, Pavel Smirnov, Felix Ocker, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Stephan Hasler, Daniel Tanneberg, and Michael Gienger. Copal: corrective planning of robot actions with large language models. In *2024 IEEE international conference on robotics and automation (ICRA)*, pages 8664–8670. IEEE, 2024.
- [16] Shyam Sundar Kannan, Vishnunandan LN Venkatesh, and Byung-Cheol Min. Smart-llm: Smart multi-agent robot task planning using large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12140–12147. IEEE, 2024.
- [17] Philip Koopman and Michael Wagner. Challenges in autonomous vehicle testing and validation. *SAE International Journal of Transportation Safety*, 4(1):15–24, 2016.

- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [19] Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Erran Li Li, Ruohan Zhang, et al. Embodied agent interface: Benchmarking llms for embodied decision making. *Advances in Neural Information Processing Systems*, 37:100428–100534, 2024.
- [20] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
- [21] Yang Liu, Weixing Chen, Yongjie Bai, Xiaodan Liang, Guanbin Li, Wen Gao, and Liang Lin. Aligning cyber space with physical world: A comprehensive survey on embodied ai. *IEEE/ASME Transactions on Mechatronics*, 2025.
- [22] Xiaoya Lu, Zeren Chen, Xuhao Hu, Yijin Zhou, Weichen Zhang, Dongrui Liu, Lu Sheng, and Jing Shao. Is-bench: Evaluating interactive safety of vlm-driven embodied agents in daily household tasks. *arXiv preprint arXiv:2506.16402*, 2025.
- [23] Chenyang Ma, Guangyu Yang, Kai Lu, Shitong Xu, Bill Byrne, Niki Trigoni, and Andrew Markham. Cyclevla: Proactive self-correcting vision-language-action models via subtask backtracking and minimum bayes risk decoding. *arXiv preprint arXiv:2601.02295*, 2026.
- [24] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Sankhyā: The Indian Journal of Statistics, Series A (2008-)*, 80:S1–S7, 2018.
- [25] Viacheslav Moskalenko, Vyacheslav Kharchenko, Alona Moskalenko, and Borys Kuzikov. Resilience and resilient systems of artificial intelligence: taxonomy, models and methods. *Algorithms*, 16(3):165, 2023.
- [26] Fei Ni, Min Zhang, Pengyi Li, Yifu Yuan, Lingfeng Zhang, Yuecheng Liu, Peilong Han, Longxin Kou, Shaojin Ma, Jinbin Qiao, David Gamaliel Arcos Bravo, Yuening Wang, Xiao Hu, Zhanguang Zhang, Xianze Yao, Yutong Li, Zhao Zhang, Ying Wen, Ying-Cong Chen, Xiaodan Liang, Liang Lin, Bin He, Haitham Bou-Ammar, He Wang, Huazhe Xu, Jiankang Deng, Shan Luo, Shuqiang Jiang, Wei Pan, Yang Gao, Stefanos Zafeiriou, Jan Peters, Yuzheng Zhuang, Yingxue Zhang, Yan Zheng, Hongyao Tang, and Jianye Hao. Embodied arena: A comprehensive, unified, and evolving evaluation platform for embodied ai, 2025.
- [27] Barak Or. Mtrr-a: Measuring cognitive recovery latency in multi-agent systems. *arXiv preprint arXiv:2511.20663*, 2025.
- [28] Amanda Prorok, Matthew Malencia, Luca Carlone, Gaurav S Sukhatme, Brian M Sadler, and Vijay Kumar. Beyond robustness: A taxonomy of approaches towards resilient multi-robot systems. *arXiv preprint arXiv:2109.12343*, 2021.
- [29] Xavier Puig, Eric Undersander, Andrew Szot, Mikael Dallaire Cote, Tsung-Yen Yang, Ruslan Partsey, Ruta Desai, Alexander William Clegg, Michal Hlavac, So Yeon Min, et al. Habitat 3.0: A co-habitat for humans, avatars and robots. *arXiv preprint arXiv:2310.13724*, 2023.
- [30] Ralf Römer, Yi Zhang, and Angela P Schoellig. Clare: Continual learning for vision-language-action models via autonomous adapter routing and expansion. *arXiv preprint arXiv:2601.09512*, 2026.
- [31] Neetesh Sharma, Armin Tabandeh, and Paolo Gardoni. Resilience analysis: A mathematical formulation to model resilience of engineering systems. *Sustainable and Resilient Infrastructure*, 3(2):49–67, 2018.
- [32] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

- [33] Marta Skreta, Zihan Zhou, Jia Lin Yuan, Kourosh Darvish, Alán Aspuru-Guzik, and Animesh Garg. Replan: Robotic replanning with perception and language models. *arXiv preprint arXiv:2401.04157*, 2024.
- [34] Xin Tan, Bangwei Liu, Yicheng Bao, Qijian Tian, Zhenkun Gao, Xiongbin Wu, Zhihao Luo, Sen Wang, Yuqi Zhang, Xuhong Wang, et al. Towards safe and trustworthy embodied ai: Foundations, status, and prospects. 2025.
- [35] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [36] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [37] David D Woods. Four concepts for resilience and the implications for the future of resilience engineering. *Reliability engineering & system safety*, 141:5–9, 2015.
- [38] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, et al. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025.
- [39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [40] Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. Safeagentbench: A benchmark for safe task planning of embodied llm agents. *arXiv preprint arXiv:2412.13178*, 2024.
- [41] Naoki Yokoyama, Sehoon Ha, and Dhruv Batra. Success weighted by completion time: A dynamics-aware evaluation criteria for embodied navigation. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1562–1569. IEEE, 2021.
- [42] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, et al. R-judge: Benchmarking safety risk awareness for llm agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1467–1490, 2024.
- [43] Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, et al. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.
- [44] Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, et al. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11142–11152, 2025.
- [45] Tao Zhang, Kaixian Qu, Zhibin Li, Jiajun Wu, Marco Hutter, Manling Li, and Fan Shi. Using large language models for embodied planning introduces systematic safety risks. *arXiv preprint arXiv:2604.18463*, 2026.
- [46] Zihao Zhu, Bingzhe Wu, Zhengyou Zhang, Lei Han, Qingshan Liu, and Baoyuan Wu. Earbench: Towards evaluating physical risk awareness for task planning of foundation model-based embodied ai agents. *arXiv preprint arXiv:2408.04449*, 2024.

462 Appendices

463	A	Practical Mechanisms Details	14
464		A.1 Rebound Metrics Detailed Calculation	14
465		A.2 Stability Metrics Detailed Calculation	14
466		A.3 Graceful Extensibility Metrics Detailed Calculation	16
467		A.4 Value Function Implementation	17
468			
469	B	Dataset Illustration	20
470		B.1 Dataset Section Components	20
471		B.2 Experiments Dataset Setting	20
472		B.3 Datasets Generation Approach	20
473		B.4 Perturbation Setting	21
474		B.5 Limitation and Data Leakage	21
475			
476	C	Resilience Evaluation Implementation	23
477		C.1 EAS Evaluation Methods Comparison	23
478		C.2 EAS Methods Overview	23
479		C.3 EAS Methods Implementation	23
480		C.4 EAS Self-Evolve Baseline Protocols	25
481		C.5 Targeted Diagnosis Optimization Implementation	26
482			
483	D	Detailed Experiment Results	29
484		D.1 Specific Resilience Aspect Analysis in Validity Experiments	29
485		D.2 Resilience Metrics Statistical Reliability	30
486		D.3 Scalability Sensitivity Analysis	31
487		D.4 Concrete Case Study	32
488		D.5 Taxonomy of Failure Modes and Resilience Signatures	35
489		D.6 Embodied Task Execution Outputs List	37
490			
491	E	LLM Usage and Prompt	38
492		E.1 LLM Usage Overview	38
493		E.2 LLM Reasoning Backbone Usage	39
494		E.3 LLM-as-a-Judge Usage	40
495		E.4 Datasets Generation LLM Usage	42
496		E.5 EAS Self-Evolve LLM Usage	43
497			
498	F	NeurIPS Paper Checklist	44
499			

A Practical Mechanisms Details

A.1 Rebound Metrics Detailed Calculation

Rebound Window Definition.

$$[t_d, t_r] \quad (11)$$

where t_d is the first step at which execution departs from the nominal continuation under the local anchor a_t , and t_r is the first step at which the task re-enters the acceptable execution. We detect the rebound window from the proposition driven by the EAS execution each episode, detector is called each LLM planner making decisions, we calculate progress and monitor whether it enters/leaves the rebound window.

Recovery Cost State Excess Vector Formalization.

$$\begin{aligned} \mathbf{z}_t^{\text{cog}} &= \left[\frac{(T_t^{\text{plan}} - \bar{T}^{\text{plan}}(a_t))_+}{\bar{T}^{\text{plan}}(a_t) + \epsilon}, \frac{(T_t^{\text{per}} - \bar{T}^{\text{per}}(a_t))_+}{\bar{T}^{\text{per}}(a_t) + \epsilon}, \frac{(T_t^{\text{coord}} - \bar{T}^{\text{coord}}(a_t))_+}{\bar{T}^{\text{coord}}(a_t) + \epsilon} \right]^\top, \\ \mathbf{z}_t^{\text{phy}} &= \left[\frac{(N_t^{\text{nav}} - \bar{N}^{\text{nav}}(a_t))_+}{\bar{N}^{\text{nav}}(a_t) + \epsilon}, \frac{(N_t^{\text{mpl}} - \bar{N}^{\text{mpl}}(a_t))_+}{\bar{N}^{\text{mpl}}(a_t) + \epsilon}, \frac{(N_t^{\text{retry}} - \bar{N}^{\text{retry}}(a_t))_+}{\bar{N}^{\text{retry}}(a_t) + \epsilon} \right]^\top, \\ \mathbf{z}_t^{\text{deb}} &= \left[\frac{(-\Delta p_t)_+}{\Delta p(a_t) + \epsilon}, \frac{(-\Delta q_t)_+}{\Delta q(a_t) + \epsilon}, \frac{(r_t - \bar{r}(a_t))_+}{\bar{r}(a_t) + \epsilon} \right]^\top. \end{aligned} \quad (12)$$

Rebound Statistics Aggregation and Signature Capture. At the *Task-Family* level, we aggregate episode-level rebound costs under the same family f and EAS self-evolve version $\pi^{(k)}$ as:

$$\bar{C}_{\text{rec},f}^{(k)} = \frac{1}{N_f^{(k)}} \sum_{i=1}^{N_f^{(k)}} C_{\text{rec}}^{(i,k)}, \quad Q_{\alpha,f}^{(k)} = \text{Quantile}_\alpha \left(\left\{ C_{\text{rec}}^{(i,k)} \right\}_{i=1}^{N_f^{(k)}} \right). \quad (13)$$

Here, $\bar{C}_{\text{rec},f}^{(k)}$ summarizes the typical rebound burden, while $Q_{\alpha,f}^{(k)}$ captures tail recovery cost under the same task-family distribution. Reporting both is important, because two agents may have similar average recovery cost but very different tail behavior.

For rebound signature capture, we track the family-level rebound profile across versions under a fixed benchmark and inner-stress protocol:

$$\mathcal{R}_{\text{reb}}^{(k)}(f) = \left(\bar{C}_{\text{rec},f}^{(k)}, Q_{\alpha,f}^{(k)} \right), \quad \Delta \mathcal{R}_{\text{reb}}^{(k)}(f) = \mathcal{R}_{\text{reb}}^{(k)}(f) - \mathcal{R}_{\text{reb}}^{(k-1)}(f). \quad (14)$$

A sustained decrease in both mean and tail rebound cost indicates improved recovery efficiency under inner stress, whereas a decrease in the mean accompanied by a persistent high tail cost suggests brittle improvement.

A.2 Stability Metrics Detailed Calculation

Our computation follows a unified pipeline: heterogeneous execution evidence is first compressed into a latent state representation, then evaluated by a critic-side value network, and finally projected to trajectory-level instability and family-level perturbation sensitivity. Here we focus on the practical symbols, computational flow, and the role of each component.

Components and Roles. We take the component description in Tabel 4.

Encoded State Representation. At each step t , we summarize heterogeneous runtime evidence into

$$\mathbf{h}_t = [\mathbf{h}_t^{\text{task}}, \mathbf{h}_t^{\text{world}}, \mathbf{h}_t^{\text{event}}, \mathbf{h}_t^{\text{prog}}], \quad (15)$$

and compute the latent state

$$s_t = E_\phi(\mathbf{h}_t). \quad (16)$$

The StateEncoder is not used to reconstruct the full world state. Instead, it provides a compact sufficient representation of the current execution context for critic-side diagnosis. This is consistent with the use of belief-like compressed states in partially observable embodied decision processes.

Component	Symbol	Role in Stability Computation
Task-context encoder	$\mathbf{h}_t^{\text{task}}$	Encodes instruction, task-family identity, and proposition-related context.
Structured world-state encoder	$\mathbf{h}_t^{\text{world}}$	Encodes objects, receptacles, rooms, agent holdings, and other structured environment state.
Execution-event encoder	$\mathbf{h}_t^{\text{event}}$	Encodes recent replanning, abnormal responses, retries, and safety-triggered mode transitions.
Progress encoder	$\mathbf{h}_t^{\text{prog}}$	Encodes task completion, proposition completion, and stepwise execution counters.
StateEncoder	E_ϕ	Compresses heterogeneous evidence into a belief-like latent state s_t .
Value network	V_ψ	Estimates critic-side remaining execution utility from the latent state.
StageBaseline	$(\mu(a_t), \Sigma(a_t))$	Provides local nominal references for the current stage context.
Instability loss	$\ell(\pi, z_i)$	Measures trajectory-level instability over an episode.
Perturbation sensitivity	β_f	Measures family-level sensitivity of instability to controlled perturbations.

Table 4: Practical components used in Stability computation.

Step	Operation
1	Extract stepwise evidence from execution, including task progress, proposition status, planner events, abnormal responses, and safety-related signals.
2	Build heterogeneous feature groups $\mathbf{h}_t^{\text{task}}, \mathbf{h}_t^{\text{world}}, \mathbf{h}_t^{\text{event}}, \mathbf{h}_t^{\text{prog}}$.
3	Encode them into a latent state $s_t = E_\phi(\mathbf{h}_t)$.
4	Evaluate critic-side progress signals $V_\psi(s_t)$, δ_t , and A_t along the trajectory.
5	Form stage-centered deviation features $\mathbf{x}_t$ and compare them with the local StageBaseline $(\mu(a_t), \Sigma(a_t))$.
6	Compute local instability deviation d_t^{stab} and aggregate it into the episode-level loss $\ell(\pi, z_i)$.
7	Compare clean and perturbed episodes within each task family to obtain the family-level perturbation sensitivity β_f .

Table 5: Practical computation flow of the Stability metric.

Stage-centered Stability Features and Loss To measure instability relative to the proper operating regime, we organize execution around the local stage anchor a_t and its corresponding StageBaseline. We define the instantaneous stability feature vector

$$\mathbf{x}_t = [\Delta V_t, \delta_t, A_t, \Delta p_t, \Delta q_t, e_t^{\text{plan}}]^\top, \quad (17)$$

where $\Delta V_t = V_\psi(s_t) - V_\psi(s_{t-1})$, Δp_t is the task-progress increment, Δq_t is the proposition-completion increment, and e_t^{plan} denotes planner-side regime events such as replanning or planning-transition indicators.

We compare $\mathbf{x}_t$ against the local nominal regime by a covariance-aware deviation:

$$d_t^{\text{stab}} = \sqrt{(\mathbf{x}_t - \mu(a_t))^\top (\Sigma(a_t) + \eta I)^{-1} (\mathbf{x}_t - \mu(a_t))}, \quad (18)$$

where $\mu(a_t)$ and $\Sigma(a_t)$ are the StageBaseline mean and covariance, and ηI is a regularizer. This form is a Mahalanobis-type deviation, which measures multivariate departure while accounting for feature correlations.

$$\ell(\pi, z_i) = \frac{1}{T_i} \sum_{t=1}^{T_i} d_t^{\text{stab}}. \quad (19)$$

539 Practical Computation Flow.

540 A.3 Graceful Extensibility Metrics Detailed Calculation

541 Graceful Extensibility (GE) measures whether an EAS preserves a minimum execution contract as
 542 the stress intensity increases. In this subsection, λ always denotes the GE stress intensity, and is
 543 unrelated to the GAE parameter in Stability. The calculation follows a simple chain:

$$m_i(\lambda) \rightarrow \mathbf{r}_f(\lambda) \rightarrow M_f(\lambda) \rightarrow \lambda_f^*$$

544 where $m_i(\lambda)$ is episode-level contract evidence, $\mathbf{r}_f(\lambda)$ is the task-family summary, $M_f(\lambda)$ is the
 545 stress margin, λ_f^* is the stress capacity.

546 **Stress Grid and Episode Evidence.** For each task family f , we evaluate the same family under a
 547 controlled stress grid

$$\Lambda = \{\lambda_1, \dots, \lambda_m\}, \quad 0 \leq \lambda_1 < \dots < \lambda_m \leq 1. \quad (20)$$

548 In our implementation, $\lambda = 0$ is the clean reference, while larger λ values instantiate stronger stress
 549 through the configured stress operator $\mathcal{T}_\lambda$.

550 For episode i in task family f under stress λ , we collect the episode-level GE evidence

$$m_i(\lambda) = (p_i(\lambda), q_i(\lambda), u_i(\lambda), c_i(\lambda), v_i(\lambda)), \quad (21)$$

551 where p_i is task completion, q_i is proposition satisfaction, u_i is safety load, c_i is rebound recovery
 552 cost, and v_i is stability load. The first two terms are benefit signals, so larger values are better. The
 553 last three terms are load signals, so smaller values are better.

554 **Contract Ratios and Episode Hard Monitor.** GE evaluates whether an execution satisfies a
 555 minimal contract over completion, proposition satisfaction, safety, recovery cost, and stability load.
 556 Let

$$\boldsymbol{\tau} = (\tau_{\text{pc}}, \tau_{\text{q}}, \tau_{\text{safe}}, \tau_{\text{rec}}, \tau_{\text{stab}})$$

557 be the corresponding thresholds. We convert each raw signal into a normalized contract ratio:

$$\begin{aligned} \rho_{i,\text{pc}}(\lambda) &= \frac{p_i(\lambda)}{\tau_{\text{pc}} + \epsilon}, & \rho_{i,\text{q}}(\lambda) &= \frac{q_i(\lambda)}{\tau_{\text{q}} + \epsilon}, \\ \rho_{i,\text{safe}}(\lambda) &= \frac{\tau_{\text{safe}}}{u_i(\lambda) + \epsilon}, & \rho_{i,\text{rec}}(\lambda) &= \frac{\tau_{\text{rec}}}{c_i(\lambda) + \epsilon}, & \rho_{i,\text{stab}}(\lambda) &= \frac{\tau_{\text{stab}}}{v_i(\lambda) + \epsilon}. \end{aligned} \quad (22)$$

558 A ratio above 1 means that the corresponding contract term passes its threshold. For numerical
 559 stability, each ratio is clipped into $[0, \rho_{\text{max}}]$ before aggregation:

$$\tilde{\rho}_{i,j}(\lambda) = \text{clip}_{[0, \rho_{\text{max}}]}(\rho_{i,j}(\lambda)), \quad j \in \{\text{pc}, \text{q}, \text{safe}, \text{rec}, \text{stab}\}. \quad (23)$$

560 The episode-level hard monitor uses the weakest contract term:

$$m_i^{\text{hard}}(\lambda) = \min_j \tilde{\rho}_{i,j}(\lambda) - 1. \quad (24)$$

561 Thus, $m_i^{\text{hard}}(\lambda) < 0$ indicates that at least one contract dimension fails for that episode.

562 **Family-level Contract Statistics.** For each task family f and stress level λ , we aggregate episode
 563 evidence using conservative quantiles. Benefit signals use the lower quantile, while load signals use
 564 the upper quantile:

$$\mathbf{r}_f(\lambda) = (p_f(\lambda), q_f(\lambda), u_f(\lambda), c_f(\lambda), v_f(\lambda)), \quad (25)$$

565 where

$$\begin{aligned} p_f(\lambda) &= Q_\alpha(\{p_i(\lambda)\}_{i \in f}), & q_f(\lambda) &= Q_\alpha(\{q_i(\lambda)\}_{i \in f}), \\ u_f(\lambda) &= Q_{1-\alpha}(\{u_i(\lambda)\}_{i \in f}), & c_f(\lambda) &= Q_{1-\alpha}(\{c_i(\lambda)\}_{i \in f}), & v_f(\lambda) &= Q_{1-\alpha}(\{v_i(\lambda)\}_{i \in f}). \end{aligned} \quad (26)$$

566 This aggregation makes GE sensitive to lower-tail task performance and upper-tail execution burden,
 567 instead of only measuring average behavior.

568 **Stress Margin and Boundary Events.** We compute family-level ratios from $\mathbf{r}_f(\lambda)$ in the same
 569 way as Eq. 22. Let

$$\tilde{\rho}_f(\lambda) = \{\tilde{\rho}_{f,\text{pc}}, \tilde{\rho}_{f,\text{q}}, \tilde{\rho}_{f,\text{safe}}, \tilde{\rho}_{f,\text{rec}}, \tilde{\rho}_{f,\text{stab}}\}$$

Step	Operation
1	Generate a stress sweep Λ with $\lambda = 0$ as the clean reference and $\lambda > 0$ as controlled stress.
2	For each rollout, extract episode evidence $h_i(\lambda)$: task completion, proposition satisfaction, safety load, rebound cost, and stability load.
3	Convert each evidence term into clipped contract ratios $\tilde{\rho}_{i,j}(\lambda)$ and compute the episode hard monitor $m_i^{\text{hard}}(\lambda)$.
4	Group valid episodes by task family f and stress level λ .
5	Aggregate lower-tail benefit and upper-tail load statistics into $\mathbf{r}_f(\lambda)$.
6	Compute the family-level soft margin $M_f(\lambda)$, hard margin $M_f^{\text{hard}}(\lambda)$, and boundary indicators $D_f(\lambda)$.
7	Aggregate the stress curve over Λ to obtain λ_f^* , AUDC_f , margin drop, degradation rate, and cliff flags.

Table 6: Practical computation flow of the Graceful Extensibility metric.

be the clipped family-level ratios. The soft GE margin is the conservative bottom- K_{GE} average:

$$M_f(\lambda) = \frac{1}{K_{\text{GE}}} \sum_{\rho \in \text{Bottom}_{K_{\text{GE}}}(\tilde{\rho}_f(\lambda))} \rho - 1. \quad (27)$$

In the implementation, this value is reported as `ge_contract_margin_min`. A positive margin means that the weakest contract dimensions still satisfy the target threshold on average; a negative margin means that the family reaches a stress boundary.

We also compute a hard margin

$$m_f^{\text{hard}}(\lambda) = \min_{\rho \in \tilde{\rho}_f(\lambda)} \rho - 1, \quad (28)$$

and derive boundary indicators:

$$\begin{aligned} D_f^{\text{soft}}(\lambda) &= \mathbb{I}[M_f(\lambda) < 0], \\ D_f^{\text{hard}}(\lambda) &= \mathbb{I}[\neg (M_f(\lambda) \geq 0 \wedge M_f^{\text{hard}}(\lambda) \geq -\delta_{\text{hard}})], \\ D_f^{\text{near}}(\lambda) &= \mathbb{I}[|M_f(\lambda)| \leq \delta_{\text{bd}}]. \end{aligned} \quad (29)$$

Here, D_f^{soft} marks contract failure, D_f^{hard} marks strict hard-gate failure, and D_f^{near} marks near-boundary behavior.

Stress Capacity and Degradation Area. Given the discrete stress grid Λ , the GE stress capacity is

$$\lambda_f^* = \max \{ \lambda \in \Lambda \mid M_f(\lambda) \geq 0 \}. \quad (30)$$

If no stress level satisfies the contract, λ_f^* is undefined for that family. This metric is meaningful only when the GE stress sweep contains multiple valid λ cells; a single unstressed run can report a cell margin, but cannot estimate formal stress capacity.

To summarize the entire degradation curve, we compute the area under the degradation-compatible curve:

$$\text{AUDC}_f = \sum_{j=2}^m \frac{y_f(\lambda_j) + y_f(\lambda_{j-1})}{2} (\lambda_j - \lambda_{j-1}), \quad y_f(\lambda) = \max(M_f(\lambda) + 1, 0). \quad (31)$$

A larger AUDC_f indicates that the EAS preserves more contract margin across the stress path. We additionally track the margin drop, maximum degradation rate, monotonicity violations, and cliff events to distinguish smooth degradation from abrupt boundary collapse.

Practical Computation Flow. In a word, we express our EAS Resilience Evaluation framework as Fig 6.

A.4 Value Function Implementation

This section describes the implementation details of the critic-side value function used by our resilience evaluation layer. The value function is not the embodied agent policy. Instead, it is an

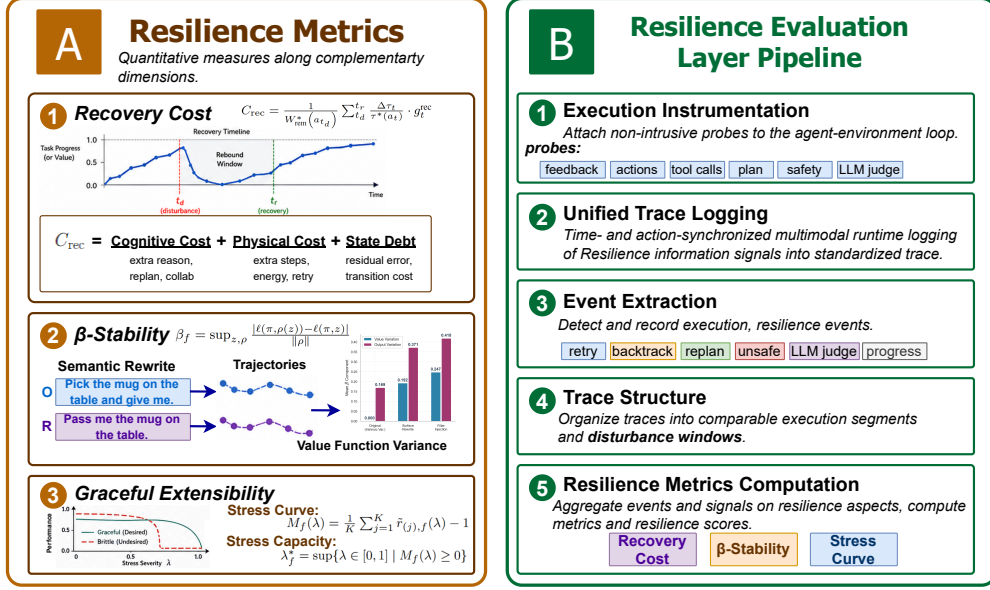


Figure 6: **Overview of our EAS Resilience Evaluation framework.** (A) illustrates resilience metrics concept and their computation, and (B) illustrates the resilience evaluation layer pipeline.

592 auxiliary evaluator that maps execution states to a scalar estimate of remaining execution utility, and
 593 provides critic-side traces for Stability and GE diagnostics.

594 **State Encoding.** At transition t , the critic does not access a privileged simulator state. Instead, it
 595 receives a critic-side observation

$$o_t^c = (\mathcal{W}_t, \mathcal{I}_t), \quad (32)$$

596 where $\mathcal{W}_t$ is the structured world-state dictionary and $\mathcal{I}_t$ is the auxiliary execution-information
 597 dictionary. The StateEncoder implements a deterministic mapping

$$E_\phi : o_t^c \mapsto s_t \in \mathbb{R}^{128}, \quad (33)$$

598 where s_t is a critic-side latent execution state, not a reconstruction of the full world state.

599 The encoder has two input branches. The textual branch first extracts a set of typed text segments

$$\mathcal{S}_t = \text{SegText}(\mathcal{W}_t, \mathcal{I}_t), \quad (34)$$

600 covering the task instruction, task objects, receptacles, rooms, object-state affordances, proposition
 601 progress, planning step, recent action, recent feedback, agent poses, held objects, and task-relevant
 602 object locations. These segments are concatenated and length-truncated into a bounded description

$$d_t = \text{Trunc}_{512}(\text{Concat}(\mathcal{S}_t)). \quad (35)$$

603 A frozen text encoder then produces

$$\mathbf{e}_t^{\text{text}} = T_\theta(d_t) \in \mathbb{R}^{384}, \quad (36)$$

604 where T_θ is the sentence-transformer encoder in the default setting. If this encoder is unavailable, the
 605 implementation uses a deterministic hash fallback, preserving reproducibility rather than introducing
 606 runtime LLM rewriting.

607 The numerical branch extracts normalized execution features

$$\mathbf{e}_t^{\text{num}} = \text{PadTrunc}_{32}(F_{\text{num}}(\mathcal{W}_t, \mathcal{I}_t)) \in \mathbb{R}^{32}. \quad (37)$$

608 F_{num} includes two-agent poses, task-progress ratio, proposition-satisfaction fraction, normalized
 609 agent-object and object-target distances, object-at-target fraction, task-object and target counts,
 610 visibility fractions, holding indicators, and normalized action-history length. Distances, counts, and
 611 action-history length are normalized by fixed constants before padding or truncation.

612 The final encoded state is obtained by feature fusion:

$$\mathbf{u}_t = [m_{\text{text}} \mathbf{e}_t^{\text{text}}; m_{\text{num}} \mathbf{e}_t^{\text{num}}], \quad (38)$$

613 where $m_{\text{text}}, m_{\text{num}} \in \{0, 1\}$ implement the configured input variant, such as full, text-only, or
614 numerical-only encoding. The fusion MLP is

$$s_t = \sigma(W_2 \text{Drop}(\text{LN}(\sigma(W_1 \mathbf{u}_t + b_1))) + b_2), \quad (39)$$

615 where σ is ReLU, LN is LayerNorm, and Drop is dropout. In the released configuration, W_1 maps
616 $384 + 32$ dimensions to 256 hidden dimensions, and W_2 maps 256 dimensions to the 128-dimensional
617 latent state.

618 **Value Network Architecture.** Given s_t , the value network estimates the critic-side remaining
619 execution utility:

$$\hat{V}_\psi(s_t) \approx \mathbb{E} \left[\sum_{u=t}^T \gamma^{u-t} \tilde{r}_u \mid s_t \right], \quad (40)$$

620 where $\tilde{r}_u$ is the reward used by the critic, including LLM-shaped reward when reward shaping is
621 enabled. The network is a three-layer MLP with hidden dimensions $[256, 128, 64]$:

$$\mathbf{h}_t^{(0)} = s_t, \quad \mathbf{h}_t^{(\ell)} = \text{Drop} \left(\text{LN} \left(\sigma \left(W_\ell \mathbf{h}_t^{(\ell-1)} + b_\ell \right) \right) \right), \quad \ell = 1, 2, 3. \quad (41)$$

622 The scalar value prediction is

$$\hat{V}_\psi(s_t) = \mathbf{w}_v^\top \mathbf{h}_t^{(3)} + b_v. \quad (42)$$

623 The implementation exposes both $\hat{V}_\psi(s_t)$ and the penultimate representation

$$\mathbf{g}_t = \mathbf{h}_t^{(3)} \in \mathbb{R}^{64}, \quad (43)$$

624 through `forward_with_features`. This representation is used for diagnostic stability traces, while
625 the scalar output is used for TD and GAE computation.

626 **Critic Target and Optimization.** For each stored transition $(s_t, a_t, \tilde{r}_t, s_{t+1}, d_t)$, the critic computes

$$\delta_t = \tilde{r}_t + \gamma(1 - d_t) \hat{V}_\psi(s_{t+1}) - \hat{V}_\psi(s_t). \quad (44)$$

627 Generalized Advantage Estimation is computed recursively as

$$A_t = \delta_t + \gamma \lambda_{\text{GAE}} (1 - d_t) A_{t+1}, \quad (45)$$

628 and the value target is

$$R_t = A_t + \hat{V}_\psi(s_t). \quad (46)$$

629 The ValueNetwork is optimized with mean-squared error:

$$\mathcal{L}_V = \frac{1}{T} \sum_{t=1}^T \left(\hat{V}_\psi(s_t) - R_t \right)^2. \quad (47)$$

630 The released implementation uses Adam with learning rate 10^{-3} , Xavier initialization for linear
631 layers, dropout rate 0.1, and gradient clipping with maximum norm 1.0.

632 **Stability Trace from the Value Network.** The value network also produces a trajectory-level sta-
633 bility trace. For each step, we combine value change, TD residual, GAE magnitude, and penultimate-
634 feature change:

$$\mathbf{o}_t = [|\Delta V_t|, |\delta_t|, |A_t|, \|\Delta \mathbf{g}_t\|_2]. \quad (48)$$

635 The implementation computes a weighted diagonal oscillation proxy

$$\ell_t^{\text{osc}} = \|\mathbf{w}_{\text{osc}} \odot \mathbf{o}_t\|_2, \quad \mathbf{w}_{\text{osc}} = [1.0, 1.0, 1.0, 0.25], \quad (49)$$

636 and averages it over the episode. This trace is not used as the final Stability metric by itself; instead,
637 it provides critic-side evidence that is later compared across clean and perturbed executions.

638 **Value Network Hyperparameter Design.** The released critic configuration uses $\gamma = 0.80$,
639 $\lambda_{\text{GAE}} = 0.65$, and an n -step horizon of 5. The StateEncoder uses text dimension 384, numeri-
640 cal dimension 32, fusion hidden dimension 256, output dimension 128, and fusion dropout 0.1. The
641 ValueNetwork uses hidden dimensions $[256, 128, 64]$, dropout 0.1, Adam learning rate 10^{-3} , and
642 gradient clipping with max norm 1.0. The default execution device is CPU with automatic CUDA

643 fallback behavior when configured. For evaluation-oriented runs, the critic can use deterministic
 644 inference mode, which switches both the StateEncoder and ValueNetwork to evaluation mode and
 645 disables dropout during measurement. Debug export records state hashes, text segments, numerical
 646 features, state vectors, value traces, TD errors, and GAE targets for auditability.

647 B Dataset Illustration

648 This section describes the dataset structure, generation protocol, perturbation setting, and evidence
 649 collected for resilience evaluation. Our code and data are available at https://anonymous.4open.science/r/EmbodiedAgentSystem_ResilienceEvaluation-0C3A/.
 650

651 B.1 Dataset Section Components

652 Our evaluation suite is built on PARTNR-style collaborative household episodes and augments them
 653 with resilience-specific execution evidence. Each episode contains a natural-language instruction, a
 654 simulator initialization, task-relevant object and receptacle records, evaluation propositions, proposi-
 655 tion dependencies, temporal or spatial constraints, and metadata for replaying the episode in Habitat.
 656 For resilience evaluation, the dataset itself is not only an outcome benchmark; it is the anchor from
 657 which we derive repeated executions under controlled perturbations and collect process-level traces.

658 We formalize an episode as

$$e_i = (\iota_i, x_i^0, \mathcal{O}_i, \mathcal{R}_i, \mathcal{P}_i, \mathcal{D}_i, \mathcal{C}_i, \mu_i), \quad (50)$$

659 where ι_i is the instruction, x_i^0 is the simulator initialization, $\mathcal{O}_i$ and $\mathcal{R}_i$ are object and receptacle
 660 records, $\mathcal{P}_i$ is the proposition set, $\mathcal{D}_i$ denotes proposition dependencies, $\mathcal{C}_i$ denotes evaluation
 661 constraints, and μ_i stores episode metadata and provenance. Runtime stress tests then generate
 662 execution artifacts

$$A_{i,\lambda,s} = (e_i, \mathcal{T}_{\lambda,s}, \tau_{i,\lambda,s}, \mathcal{L}_{i,\lambda,s}, \mathcal{C}_{i,\lambda,s}, \mathcal{J}_{i,\lambda,s}), \quad (51)$$

663 where $\mathcal{T}_{\lambda,s}$ is the perturbation operator with stress level λ and seed s , τ is the trajectory, $\mathcal{L}$ is the
 664 runtime log, $\mathcal{C}$ is the monitor state, and $\mathcal{J}$ is the LLM-judge or critic-side signal. This separation is
 665 important: perturbations are applied as runtime operators over fixed episode identities, rather than as
 666 unrelated new datasets.

667 B.2 Experiments Dataset Setting

668 The experiment suite defines, for each research question, how the validation pool is sampled, how
 669 perturbations are applied, and how runtime evidence is aggregated. Validation Experiments (Exp-Q1)
 670 verifies whether the metrics expose non-outcome process differences. Benchmark Experiments
 671 (Exp-Q2) uses the same instrumentation as a benchmark protocol across EAS methods and stress
 672 regimes. Optimization Experiments (Exp-Q3) uses metric diagnosis to compare original and opti-
 673 mized execution stacks under matched episode identities.

674 The dominant proposition types include `is_on_top`, `is_next_to`, `is_in_room`, `is_inside`,
 675 `is_clean`, and `is_powered_on`. This mix is useful for resilience analysis because failures can arise
 676 from different sources: spatial search, object placement, human-robot coordination, object-state
 677 predicates, and stale world-graph grounding.

678 B.3 Datasets Generation Approach

679 The dataset generation pipeline converts language-level task ideas into simulator-replayable and
 680 metric-auditable embodied episodes. Following the PARTNR-style task construction protocol [6], we
 681 generate household collaboration tasks with object-state requirements, spatial relations, and temporal
 682 or dependency constraints. We additionally preserve perturbation anchors in the task semantics so
 683 that the same episode identity can later be executed under clean and stressed conditions. The process
 684 is organized as a four-stage pipeline:

$$\iota \xrightarrow{\text{Parse}} z \xrightarrow{\text{Instantiate}} e \xrightarrow{\text{SynthesizeEval}} \tilde{e} \xrightarrow{\text{PackVerify}} \bar{e}, \quad (52)$$

685 where ι is a candidate instruction, z is a structured initialization specification, e is a replayable
 686 embodied episode, $\tilde{e}$ is the episode paired with auditable evaluation logic, and $\bar{e}$ is the packed and
 687 simulator-verified executable episode.

Table 7: Dataset and runtime evidence used by the experiment suite.

Evidence Source	Scale / Scope	Role in the paper
Full validation pool	369 episodes	Main validation-style pool used by the execution scripts through the dataset path argument.
Sensitive mechanism subset	193 episodes	Cleaner mechanism-study subset used when repeated controlled comparisons are more important than broad coverage.
Validation Exp setting	45 episodes, 3 epochs, 5 stress intensities	Metric validity subsets for rebound, stability, and GE, selected from the validation pool.
Benchmark Exp setting	369 episodes	Benchmark and stress-proxy subsets; outer stress sweeps use repeated seeds and multiple perturbation intensities.
Optimization Exp setting	45 episodes, 3 epochs, 6 comparisons	Diagnosis-driven optimization subsets comparing baseline and optimized execution stacks.
Runtime output artifacts	per spec and per episode	Runtime evidence: resolved configuration, episode metrics, planner logs, statistics, critic exports, rebound/stability/degradation reports, and margin summaries.

Stage I: instruction proposal and filtering. Instruction generation follows a templated LLM prompting workflow. The templates emphasize object states, receptacle relations, and household collaboration goals; optional clutter and fixed scene pools are used to increase diversity while keeping the sampling protocol reproducible. Generated instructions are parsed into structured initialization specifications and filtered to remove invalid, duplicate, or underspecified tasks before simulator instantiation.

Stage II: simulator episode instantiation. The instantiation stage binds each structured task to a concrete scene. It samples the required objects and receptacles, places them under the initialization constraints, applies object-state and articulated-object predicates, and stores the semantic mapping needed for later evaluation. The resulting episode records the scene, initial object layout, target objects or receptacles, object-state snapshots, and provenance metadata.

Stage III: evaluation synthesis and audit. The evaluation-synthesis stage converts each instruction into a proposition set and its associated constraints. It constructs spatial, object-state, temporal, dependency, and tie constraints, then stores an auditable intermediate representation before final packing. This intermediate representation is useful because proposition semantics can be inspected and corrected before they become executable evaluation targets.

Stage IV: packing, dependency attachment, and verification. The packing stage combines simulator initialization and evaluation semantics into executable episodes. It attaches proposition dependencies, checks within-set consistency, merges scene-specific subsets when required, and verifies that the final episodes can be replayed by the simulator. The final export is a compressed dataset used directly by the experiment runners.

B.4 Perturbation Setting

Perturbation application protocol. The perturbation injector is opt-in and idempotent: each perturbation is applied at most once for a given episode, seed, and perturbation kind. Prompt-level perturbations are applied before the LLM planner builds its prompt, while world-level perturbations modify the planner-facing World Graph snapshot. This separation lets us test language sensitivity and embodied-state sensitivity without changing the base episode identity or evaluation propositions.

B.5 Limitation & Data Leakage

Threats to Metrics Validity Construct validity. Rebound, stability, and graceful extensibility are abstract system properties. We reduce construct ambiguity by grounding every metric in observable

Table 8: Dataset generation artifacts and reproducibility roles.

Artifact	Purpose
Candidate instructions and initialization specs	Natural-language task proposals paired with structured scene, object, receptacle, and object-state constraints.
Replayable episode records	Simulator initialization with scene identity, object layout, target/receptacle annotations, object states, and provenance.
Auditable evaluation specifications	Proposition, temporal, dependency, and tie-constraint semantics inspected before packing.
Packed and verified dataset	Executable episodes with proposition constraints and simulator verification applied.
Runtime evidence outputs	Perturbation-specific traces and metrics collected without changing the base episode identity.

Table 9: Perturbation taxonomy used by the resilience evaluation suite.

Perturbation	Level	Operation Form	Evaluation Role
Clean reference	Reference	No mutation; keeps the original instruction and world state.	Provides the $\lambda = 0$ reference for comparing stressed executions.
Surface rewrite	Prompt	Deterministic synonym and wording rewrite, e.g., replacing action or instruction ordering while preserving the goal.	Tests whether the planner remains stable under semantic-equivalent instruction variation.
Filler injection	Prompt	Adds benign task-irrelevant language such as cautionary or procedural filler.	Tests sensitivity to prompt distraction without changing the task contract.
Object-state toggle	World state	Toggles a boolean object-state predicate on a task-relevant object when available.	Creates object-state or tool-precondition mismatch for Rebound analysis.
Room shift	World state	Adds a counterfactual room or allowed-region signal to the World Graph.	Tests grounding robustness under room-level ambiguity.
Holding-state toggle	World state	Changes the holding state of a task object when the World Graph exposes the corresponding mutator.	Tests manipulation-state mismatch and coordination robustness.
Irrelevant distractors	Stress budget	Injects non-goal distractor entities, with distractor count increasing with stress intensity λ .	Provides the controlled stress axis for Graceful Extensibility and boundary behavior.

718 trace events: recovery windows, replanning counts, value variance, perturbation-neighborhood
719 differences, stress margins, and lower-bound violations. LLM judgments are used as semantic
720 sensors, not as the only source of truth.

721 **Judge bias.** LLM-as-a-Judge can be sensitive to prompt wording and output format. We mitigate
722 this risk through fixed rubrics, low-temperature generation, JSON-mode structured outputs, short
723 trajectory windows, and saved critic exports. The analysis is auditable because each critic report
724 records failure points, root causes, suggestions, and execution-quality scores.

725 **Reliability.** Some resilience metrics, especially recovery cost and MTBF, are heavy-tailed because a
726 few episodes dominate failure time. We therefore report confidence intervals, repeated seeds, and
727 perturbation-conditioned summaries instead of relying only on means.

Table 10: Stress-severity schedule and intended use.

Setting	Stress Values	Purpose
Clean reference	$\lambda = 0$	Baseline execution without perturbation.
Single perturbation tests	$\lambda = 1.0$	Used for prompt rewrites and object-state changes when the goal is to compare clean vs. perturbed executions.
Reported stress heatmap	$\lambda \in \{0.0 - 1.0\}$	Summarizes how risk varies across perturbation types and stress severity.
Formal GE sweep	$\lambda \in \{0.0 - 1.0\}$ when fully enabled	Estimates the stress-response curve $M_f(\lambda)$ and stress capacity λ_f^* .

External validity. The current implementation is grounded in Habitat-Sim and PARTNR-style household tasks. Thresholds such as safe recovery cost or stress-bound margins may require recalibration for real robots, other simulators, or different tool APIs. The released contribution is therefore the measurement instrument and trace schema, not a universal numeric threshold.

Data Leakage and Contamination Risks Two leakage channels are particularly salient in our setting. First, **LLM-as-a-Judge contamination**: if the judge context overlaps with test tasks or tool APIs in our semantic GAE settings, the evaluation may overestimate stability and understate extensibility.

Second, **evolve-test entanglement**: self-evolution can inadvertently adapt to the test distribution if evolution experience is not strictly separated from held-out evaluation tasks, inflating guarantee signals.

Limitation: Cost, Throughput, and Instrumentation Probe-based monitoring and LLM judging incur overhead (latency, tool-call budget, and compute), we makes these costs explicit in traces, enabling *cost-aware* resilience assessment with trading faster rebound against higher recovery cost. To scale evaluation, we take two-stage protocol: run low-cost probes online for all episodes, and apply expensive semantic judging to (i) flagged episodes like early-warning triggers and (ii) a hierarchical sample for statistical represent. This preserves benchmark fidelity while keeping evaluation cost predictable.

C Resilience Evaluation Implementation

C.1 EAS Evaluation Methods Comparison

We analyse existing benchmarks and show the comparison in Table 11.

C.2 EAS Methods Overview

We report comparative EAS methods in Table 12.

C.3 EAS Methods Implementation

We implement the compared EAS methods as planner-side overlays on a shared embodied execution path, rather than as separate simulators or different task definitions. The common path is `planner_demo_*.py` $\rightarrow$ `EvaluationRunner.run_instruction` $\rightarrow$ `CentralizedLLMPlanner.prepare_prompt` $\rightarrow$ `LLMPlanner.replan`. The launcher fixes the Hydra configuration, dataset split, perturbation mode, and output directory; the evaluation runner resets the environment, invokes the planner, executes high-level skills, and records observations; the centralized planner serializes the world graph into room, furniture, object, and agent-state evidence; and `LLMPlanner` hosts the method-specific optimization hooks. This design keeps the semantic task, simulator, skill set, and execution budget comparable across methods, while allowing each EAS mechanism to modify only the planning procedure or the injected context.

Table 11: **Positioning existing EAS evaluation against resilience metrics.** Existing benchmarks increasingly move beyond final success, but they still provide limited process-level measurements for recovery burden, decision stability, and stress-response degradation.

Evaluation line	Representative works	Main signal	Missing resilience signal
Outcome / capability	PARTNR, EmbodiedBench	Success, completion, planning / action capability	Cannot distinguish smooth success from costly recovery.
Safety / hazard	SafeAgentBench, HazardArena	Hazardous progression, safety	Limited of post-disruption recovery and stability.
Reliability / perturbation	IS-Bench, adversarial stress tests	Performance drop under input, scene, or policy perturbations	Degradation endpoint, no recovery trajectory.
Our Resilience	Resilience Framework	Evaluation Recovery cost, β-stability, stress-response curve	Measures rebound, stability, and graceful extensibility as process properties.

Table 12: EAS resilience-relevant comparative methods and their target aspects based on our resilience aspects.

EAS Mechanisms	Typical Methods	Target Aspect
LLM planning	ReAct, SayCan [39, 2]	Reason-Action, outcome baselines
Feedback-aware	InnerMonologue, CoPAL [14, 15]	Recovery under closed-loop feedback
Reflection / memory	Reflexion, CycleVLA [32, 23]	Self-correction and oscillation
Modular / decomposition	SMART-LLM [16]	Stable consistency under decomposition
Evolve / adaptive	AgentEvolver, CLARE [43, 30]	Inner-stress and graceful extensibility

ReAct. ReAct is implemented as the direct reason-action baseline. `ZeroShotReactPlanner` constrains each LLM response to a free-form thought followed by graph-grounded tool calls and an `Assigned!` terminator, while `CentralizedLLMPlanner.prepare_prompt` supplies the current world graph as room, furniture, object, and agent-state descriptions. The optimization is therefore not a new controller, but a prompt-and-parser interface that keeps LLM reasoning aligned with executable skill names and exact graph entities.

SayCan. SayCan is implemented as candidate-level feasibility optimization. `SayCanPlanner` asks the LLM for top- K action candidates with confidence scores, `CandidateScorer` computes a fused score $s(a) = s_{\text{Say}}(a) \cdot s_{\text{Can}}(a)$ using the affordance model, and `StabilityMonitor` rejects low-score candidates through a configured fallback action. This optimizes the action-selection layer by discouraging high-likelihood but low-affordance actions, and records the Say/Can split for downstream stability analysis.

Inner Monologue. Inner Monologue is implemented as closed-loop feedback injection after skill execution. `FeedbackGenerator` combines success detection, scene description, self-state reporting, and optional critic/rebound feedback into a structured feedback block, which is inserted into the planner prompt before the next Thought. The optimization target is local recovery: the LLM receives concise environment feedback without changing the world graph, skill API, or evaluation objective.

CoPAL. CoPAL is implemented as corrective planning context triggered by recent failures. `ErrorTranslator` maps tool errors such as unreachable targets, missing objects, or precondition violations into natural-language failure causes, while `CoPALContextBuilder` combines the failed action, recent history, and a bounded recovery hint. This optimizes rebound behavior at the prompt level by turning raw simulator/tool errors into compact corrective constraints before replanning.

Reflexion. Reflexion-style self-correction is implemented through the rebound reflection tool rather than persistent model updates. `TaskReflectTool` emits a reflection trigger when the agent is